

```
// _____< START >- _____
// FILE NAME: : README.md
// TYPE      : .MD
// PATH      : README.md
// PROCESSED: 2026-09-06T21:55:35.363Z
// _____< START >- _____
(packages/home/src/assets/pendex-github-banner.png)
```

Pendex (`px`)

> ****Your project's portable codex.****

Pendex turns an entire source tree into portable, plain-text archives —ideal for AI prompts, code reviews, backups, and sharing. Compile a project into banner-delimited `.txt` files grouped by `job` (source, web, style, terminal, configuration, documentation, testing, misc), then reconstruct the whole tree later with `px split`.

Once compiled, a project becomes something you can paste into an LLM context window, attach to an email, diff against an earlier snapshot, or archive independently of git —a single, portable, human-readable representation of a codebase that can be read as plain text and reconstructed on demand.

Built on [Bun](<https://bun.sh>) + TypeScript, using [`@clack/prompts`](<https://www.npmjs.com/package/@clack/prompts>) for the interactive shell.

About the name

Pendex comes from two sources: the mathematical notation for a **power set** — $\mathcal{P}(A)$, the set of every subset of a set —and the Latin roots behind words like `_index_` and `_codex_` ("one who points out"; "book, collection"). Neither is literal —Pendex doesn't archive every possible subset of a project's files —but the name is meant to carry that same sense of `_totality_`: not a compressed export, but a complete, canonical textual representation of a project that can be explored, shared, and reconstructed.

Mathematics	Pendex	
-----	-----	
Set	Project	
Elements	Files	
Power set	Complete representation of every file/group	
Mapping	<code>`manifest.json`</code>	
Reconstruction	<code>`px split`</code>	

That framing is also why the `architecture` (see below) is split the way it is —`ArchiveFormat` (representation), `FileScanner` (discovery), `CompileService`/`SplitService` (orchestration), `CompileView`/`SplitView` (presentation) —each piece has exactly one well-defined responsibility, the same way each piece of a formal system does.

Install

```
```bash
bun install
```
```

Usage

Interactive shell

```
```bash
bun run start
```
```

Presents a menu: ****Compile Codebase****, ****Split Archive****, ****Exit Program****.

Standalone commands

``compile`` and ``split`` don't **require** the interactive shell —each only needs a ``Config``, which it loads itself via ``ConfigManager`` when run directly:

```
```bash
bun run compile # bun run src/commands/Compile.ts
bun run split # bun run src/commands/Split.ts
```
```

Compiled binary

```
```bash
bun run build # outputs ./dist/px
./dist/px
```
```

Architecture —layers, not files

Every non-trivial command **is** split into three layers **with** one job each. The mental model **is** deliberately shaped:

| Layer | Folder | Owns |
|--------------------|------------------------------|--|
| **Core** | <code>`src/core/`</code> | business logic: glob resolution, manifest building, archive read/write |
| **View** | <code>`src/views/`</code> | terminal rendering: intro/progress/summary |
| **Command** | <code>`src/commands/`</code> | <code>`ICommand`</code> <code>identity</code> (<code>`key`/`label`/`hint`</code>), wiring a View to its deps |

```
```mermaid
classDiagram
 direction TB

 class ICommand {
 <<interface>>
 +key: string
 +label: string
 +hint: string
 +execute() Promise~void~
 }

 class Compile {
 <<Command —container>>
 -theme: Theme
 -config: Config
 -view: CompileView
 +execute() Promise~void~
 }

 class Split {
 <<Command —container>>
 -theme: Theme
 -config: Config
 -view: SplitView
 }

 Compile ..|> ICommand
 Compile --> CompileView

 Split ..|> ICommand
 Split --> SplitView
```
```

```

        +execute() Promise~void~
    }
Split ..|> ICommand
Split --> SplitView

class Exit {
    <<Command —no service/view split; see Exit.ts comment>>
    +execute() Promise~void~
}
Exit ..|> ICommand

class CompileView {
    <<View —the only @clack/prompts caller for compile>>
    +render(theme, config) Promise~void~
}
CompileView --> CompileService
CompileView --> FileScanner : findEmptyDirectories

class SplitView {
    <<View —the only @clack/prompts caller for split>>
    +render(theme, config) Promise~void~
}
SplitView --> SplitService

class CompileService {
    <<Core —pure logic, no rendering>>
    +resolveGlobalExcludes(config) Promise~string[]~
    +prepareOutputDirectory(outputDir) Promise~void~
    +compileJob(job, excludes, outputDir, manifest, claimed)
Promise~CompileJobOutcome~
    +writeManifest(outputDir, manifest) Promise~void~
    +runCompile(config) Promise~CompileSummary~
}
CompileService --> FileScanner
CompileService --> ArchiveFormat

class SplitService {
    <<Core —pure logic, no rendering>>
    +readManifest(outputDir) Promise~Manifest?~
    +prepareRebuildDirectory(rebuiltDir) Promise~void~
    +splitArchiveFile(outputDir, rebuiltDir, filename)
Promise~SplitFileOutcome~
    +runSplit(outputDir, rebuiltDir) Promise~SplitSummary~
}
SplitService --> ArchiveFormat

class FileScanner {
    <<Core —glob resolution, no I/O writes>>
    +loadIgnorePatterns(path) Promise~string[]~
    +resolveJobFiles(job, excludes, claimed) Promise~string[]~
    +findEmptyDirectories(cwd, excludes) Promise~string[]~
}

class ArchiveFormat {
    <<Core —single source of truth for the archive text format>>
    +buildArchiveEntry(path, content) string
    +joinArchiveEntries(entries) string
    +splitArchiveEntries(rawText) string[]
    +parseArchiveEntry(segment) ArchivedFile?
}

class ConfigManager {
    <<Singleton>>

```

```

    +static getInstance() Promise~ConfigManager~
    +get() Config
    +reload() Promise~Config~
    +save() Promise~void~
  }

  class Application {
    <<root —thin, intentionally unsplit>>
    +init(exitFn) Promise~void~
    +run(exitFn) Promise~void~
  }
  Application "1" o-- "" ICommand
  Application --> ConfigManager

  class ThemePalette {
    <<data —Dracula colors, DefaultTheme objects>>
  }
  class useTheme {
    <<mechanism —Proxy-based chaining, palette-agnostic>>
  }
  Theme --> ThemePalette
  Theme --> useTheme
}

```

Why `this` split, and why not everywhere

- `ArchiveFormat.ts` exists because `Compile` and `Split` used to each have half-knowledge of the same text format —banners were `_written_` in one file and `_parsed_` with separately-maintained regexes in another. That's the exact failure mode SoC prevents: two places that must agree on one thing, with nothing enforcing it. Now there's one file that owns the format in both directions.
- `CompileService` / `SplitService` never `import` `@clack/prompts`. That's the actual test of the boundary —if a service needed the terminal to do its job, the split wouldn't be real. It also means `runCompile()` / `runSplit()` work `headlessly` (used by the standalone `import.meta.main` entry points) without dragging along interactive-only rendering.
- `Exit.ts` is deliberately `_not_ split` into `command/service/view`. It's one `outro()` call and a `process.exit()` —three files for that would be separation for its own sake. See the comment at the top of `Exit.ts`.
- `Application` (`App.ts`) is deliberately `_not_ split` either. It's the root —in the React analogy, the thing that mounts everything `else` —and its only real `behavior` (the menu select-loop) is inherently both state and render at once. Forcing that apart would add indirection without adding clarity.
- `ThemePalette.ts` / `useTheme.ts` split the same way: `palette` (`_what_` colors mean) from the Proxy-based chaining `engine` (`_how_` composition works). `useTheme.ts` doesn't know what a color is; `ThemePalette.ts` doesn't know how chaining works. This also means swapping `picocolors` for a custom `Color` class later only touches `ThemePalette.ts`.

Configuration

Defaults live in `[src/config/config.toml](./src/config/config.toml)`:

- `outputDir` / `rebuiltDir` —where `compile` writes archives and `split` rebuilds into, respectively.
- `exclude` —glob patterns applied globally, on top of `.gitignore`.
- `[[jobs]]` —one block per output `.txt` file: `filename`, `description`, `include` globs, and job-specific `exclude` globs. `**A`

job with an empty ``include`` list is a `_remainder_` job** —it catches every file no earlier job claimed (see ``8_MISC_FILES``) —and must stay last in the array.

At runtime, an optional ``runtime.config.json`` in the project root overrides any of the above. There's no interactive settings editor; edit ``runtime.config.json`` (or ``config.toml`` for the shipped defaults) directly.

``ConfigManager`` (``src/components/Config.ts``) is a singleton: it parses ``config.toml`` once per process, merges ``runtime.config.json`` on top if present, and hands back the same ``Config`` object to every command.

Conventions

- **No ``any``.
- The whole tree type-checks under ``strict`` with zero explicit-``any`` usage.
- **Bun over Node.
- File I/O, globbing, TOML parsing, and shelling out all use Bun's built-ins (``Bun.file``, ``Bun.write``, ``Bun.Glob``, ``Bun.$``, ``Bun.TOML``); there are no ``node:*`` imports anywhere in ``src/``. Path manipulation (``joinPath``/``dirname``/``basename``/``extName`` in ``Constants.ts``) is implemented with small local string helpers instead of ``node:path`` for the same reason.- **User-facing strings live as a ``STRINGS`` field on the class that uses them

**, not in a shared strings module. Every command and view has its own ``private readonly STRINGS = { ... } as const`` at the top — the tradeoff is intentional: it favors "one obvious place to edit this copy" over textbook copy/logic separation.

Header comments

``src/utils/HeaderComments.ts`` is a **separate, standalone utility** — not part of the ``px`` command set, and out of scope for the layered refactor above —that injects or strips a

```
````typescript
// FILE-PATH: <path>
````
```

comment at the top of project files. Run it directly:

```
````sh
bun run src/utils/HeaderComments.ts
````
```

Project structure

```
````sh
src/
 index.ts entry point
 types.d.ts cross-layer shared types
 components/
 App.ts root orchestrator (intentionally unsplit)
 Config.ts ConfigManager singleton
 Constants.ts generic app-wide constants + path helpers
 Theme.ts barrel: composes ThemePalette + useTheme
 ThemePalette.ts theme data (colors, DefaultTheme objects)
 useTheme.ts theme mechanism (Proxy-based chaining)
 commands/ Command layer —ICommand identity + wiring
 Command.ts
 Compile.ts
 Split.ts
 Exit.ts (intentionally unsplit —see file comment)
```

```
core/ Core layer —pure logic, no @clack/prompts
 ArchiveFormat.ts archive text format (write + parse)
 FileScanner.ts glob resolution, empty-dir detection
 CompileService.ts compile orchestration
 SplitService.ts split orchestration
views/ View layer —all terminal rendering
 CompileView.ts
 SplitView.ts
config/
 config.toml default Config
utils/ standalone, out of scope for this refactor
 HeaderComments.ts
 update-config.ts
 validators.ts
...

```

```
// _____< END >_____
// FILE NAME: : README.md
// TYPE : .MD
// PATH : README.md
// PROCESSED: 2026-09-06T21:55:35.363Z
// _____< END >_____
```

```
// < START >-
// FILE NAME: : README.md
// TYPE : .MD
// PATH : packages/color/README.md
// PROCESSED: 2026-09-06T21:55:35.363Z
// < START >-
```

## # How Colors Is Used

`'Colors'` is designed to be a highly performant, zero-dependency utility for formatting terminal text with standard ANSI and 24-bit Truecolor escape sequences.

## ## Basic Text Styling

To style text, pass your string directly into any of the `static` styler properties:

```

`ts
import { Colors } from './Colors';

console.log(Colors.red('This text is red!'));
console.log(Colors.bold('This text is bold!'));
`

```

## ## Nested Styling

Just like `'picocolors'`, nesting styles of the same family is fully supported. If a nested style is found, the `class` dynamically replaces the closing tag so that styling resumes correctly afterwards:

```

`ts
// Inside the red text, "bold and underlined" is nested
console.log(Colors.red('Normal red ${Colors.underline('underlined
red')} back to normal red'));
`ts

```

```
24-Bit Truecolor (Hex & bgHex)
```

You can dynamically create RGB foreground and background stylers using hex codes:

```
``ts
const customOrange = Colors.hex('#ff8800');
const customBg = Colors.bgHex('333'); // Leading # is optional
```

```
console.log(customOrange('Custom 24-bit orange text'));
console.log(customBg('Dark grey background text'));
``
```

## ## Environment Controls

You can programmatically force ANSI styling on or off regardless of the detected terminal environment:

```
``ts
Colors.disable(); // All stylers will now return plain, unmodified strings
Colors.enable(); // Force colors back on
``
```

```
// _____< END >- _____
// FILE NAME: : README.md
// TYPE : .MD
// PATH : packages/color/README.md
// PROCESSED: 2026-09-06T21:55:35.363Z
// _____< END >- _____
```

```
// _____< START >- _____
// FILE NAME: : LICENSE.md
// TYPE : .MD
// PATH : LICENSE.md
// PROCESSED: 2026-09-06T21:55:35.363Z
// _____< START >- _____
```

GNU Affero General Public License  
=====

\_Version 3, 19 November 2007\_  
\_Copyright © 2026 Dustin Dew <phyxashun@gmail.com>  
<https://phyxashun.github.io/pendex/>

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

### Preamble

The GNU Affero General Public License is a free, copyleft license for software and other kinds of works, specifically designed to ensure cooperation with the community in the case of network server software.

The licenses for most software and other practical works are designed to take away your freedom to share and change the works. By contrast, our General Public Licenses are intended to guarantee your freedom to share and change all versions of a program--to make sure it remains free software for all its users.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for them if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs, and that you know you can do these things.

Developers that use our General Public Licenses protect your rights

with two steps: **(1)** assert copyright on the software, and **(2)** offer you **this** license which gives you legal permission to copy, distribute and/or modify the software.

A secondary benefit of defending all users' freedom is that improvements made in alternate versions of the program, if they receive widespread use, become available for other developers to incorporate. Many developers of free software are heartened and encouraged by the resulting cooperation. However, in the case of software used on network servers, this result may fail to come about. The GNU General Public License permits making a modified version and letting the public access it on a server without ever releasing its source code to the public.

The GNU Affero General Public License is designed specifically to ensure that, in such cases, the modified source code becomes available to the community. It requires the operator of a network server to provide the source code of the modified version running there to the users of that server. Therefore, public use of a modified version, on a publicly accessible server, gives the public access to the source code of the modified version.

An older license, called the Affero General Public License and published by Affero, was designed to accomplish similar goals. This is a different license, not a version of the Affero GPL, but Affero has released a new version of the Affero GPL which permits relicensing under this license.

The precise terms and conditions for copying, distribution and modification follow.

## **\_TERMS AND CONDITIONS\_**

### **Definitions**

“This License” refers to version 3 of the GNU Affero General Public License.

“Copyright” also means copyright-like laws that apply to other kinds of works, such as semiconductor masks.

“The Program” refers to any copyrightable work licensed under this License. Each licensee is addressed as “you”. “Licensees” and “recipients” may be individuals or organizations.

To “modify” a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a “modified version” of the earlier work or a work “based on” the earlier work.

A “covered work” means either the unmodified Program or a work based on the Program.

To “propagate” a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying, distribution (with or without modification), making available to the public, and in some countries other activities as well.

To “convey” a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through

a computer network, with no transfer of a copy, is not conveying.

An interactive user interface displays “Appropriate Legal Notices” to the extent that it includes a convenient and prominently visible feature that **(1)** displays an appropriate copyright notice, and **(2)** tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees may convey the work under this license, and how to view a copy of this License. If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion.

#### Source Code

-----

The “source code” for a work means the preferred form of the work for making modifications to it. “Object code” means any non-source form of a work.

A “Standard Interface” means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language.

The “System Libraries” of an executable work include anything, other than the work as a whole, that **(a)** is included in the normal form of packaging a Major Component, but which is not part of that Major Component, and **(b)** serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A “Major Component”, in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.

The “Corresponding Source” for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to control those activities. However, it does not include the work's System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

The Corresponding Source need not include anything that users can regenerate automatically from other parts of the Corresponding Source.

The Corresponding Source for a work in source code form is that same work.

#### Basic Permissions

-----

All rights granted under this license are granted for the term of copyright on the Program, and are irrevocable provided the stated conditions are met. This License explicitly affirms your unlimited permission to run the unmodified Program. The output from running a covered work is covered by this License only if the output, given its content, constitutes a covered work. This License acknowledges your

rights of fair use or other equivalent, as provided by copyright law.

You may make, run and propagate covered works that you do not convey, without conditions so long as your license otherwise remains in force. You may convey covered works to others for the sole purpose of having them make modifications exclusively for you, or provide you with facilities for running those works, provided that you comply with the terms of this license in conveying all material for which you do not control copyright. Those thus making or running the covered works for you must do so exclusively on your behalf, under your direction and control, on terms that prohibit them from making any copies of your copyrighted material outside their relationship with you.

Conveying under any other circumstances is permitted solely under the conditions stated below. Sublicensing is not allowed; section 10 makes it unnecessary.

#### Protecting Users' Legal Rights From Anti-Circumvention Law

---

No covered work shall be deemed part of an effective technological measure under any applicable law fulfilling obligations under article 11 of the WIPO copyright treaty adopted on 20 December 1996, or similar laws prohibiting or restricting circumvention of such measures.

When you convey a covered work, you waive any legal power to forbid circumvention of technological measures to the extent such circumvention is effected by exercising rights under this license with respect to the covered work, and you disclaim any intention to limit operation or modification of the work as a means of enforcing, against the work's users, your or third parties' legal rights to forbid circumvention of technological measures.

#### Conveying Verbatim Copies

---

You may convey verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice; keep intact all notices stating that this license and any non-permissive terms added in accord with section 7 apply to the code; keep intact all notices of the absence of any warranty; and give all recipients a copy of this License along with the Program.

You may charge any price or no price for each copy that you convey, and you may offer support or warranty protection for a fee.

#### Conveying Modified Source Versions

---

You may convey a work based on the Program, or the modifications to produce it from the Program, in the form of source code under the terms of section 4, provided that you also meet all of these conditions:

```
> **a)** The work must carry prominent notices stating that you modified
> it, and giving a relevant date.
>
> **b)** The work must carry prominent notices stating that it is
> released under this license and any conditions added under section 7.
> This requirement modifies the requirement in section 4 to
> “keep intact all notices”.
>
```

> **\*\*c)\*\*** You must license the entire work, as a whole, under this  
 > License to anyone who comes into possession of a copy. This  
 > License will therefore apply, along with any applicable section 7  
 > additional terms, to the whole of the work, and all its parts,  
 > regardless of how they are packaged. This License gives no  
 > permission to license the work in any other way, but it does not  
 > invalidate such permission if you have separately received it.  
 >  
 > **\*\*d)\*\*** If the work has interactive user interfaces, each must display  
 > Appropriate Legal Notices; however, if the Program has interactive  
 > interfaces that do not display Appropriate Legal Notices, your  
 > work need not make them do so.

A compilation of a covered work with other separate and independent works, which are not by their nature extensions of the covered work, and which are not combined with it such as to form a larger program, in or on a volume of a storage or distribution medium, is called an “aggregate” if the compilation and its resulting copyright are not used to limit the access or legal rights of the compilation's users beyond what the individual works permit. Inclusion of a covered work in an aggregate does not cause this License to apply to the other parts of the aggregate.

#### Conveying Non-Source Forms

-----

You may convey a covered work in object code form under the terms of sections 4 and 5, provided that you also convey the machine-readable Corresponding Source under the terms of this License, in one of these ways:

> **\*\*a)\*\*** Convey the object code in, or embodied in, a physical product  
 > (including a physical distribution medium), accompanied by the  
 > Corresponding Source fixed on a durable physical medium  
 > customarily used for software interchange.  
 >  
 > **\*\*b)\*\*** Convey the object code in, or embodied in, a physical product  
 > (including a physical distribution medium), accompanied by a  
 > written offer, valid for at least three years and valid for as  
 > long as you offer spare parts or customer support for that product  
 > model, to give anyone who possesses the object code either **\*(1)\*\*** a  
 > copy of the Corresponding Source for all the software in the  
 > product that is covered by this License, on a durable physical  
 > medium customarily used for software interchange, for a price no  
 > more than your reasonable cost of physically performing this  
 > conveying of source, or **\*(2)\*\*** access to copy the  
 > Corresponding Source from a network server at no charge.  
 >  
 > **\*\*c)\*\*** Convey individual copies of the object code with a copy of the  
 > written offer to provide the Corresponding Source. This  
 > alternative is allowed only occasionally and noncommercially, and  
 > only if you received the object code with such an offer, in accord  
 > with subsection 6b.  
 >  
 > **\*\*d)\*\*** Convey the object code by offering access from a designated  
 > place (gratis or for a charge), and offer equivalent access to the  
 > Corresponding Source in the same way through the same place at no  
 > further charge. You need not require recipients to copy the  
 > Corresponding Source along with the object code. If the place to  
 > copy the object code is a network server, the Corresponding Source  
 > may be on a different server (operated by you or a third party)  
 > that supports equivalent copying facilities, provided you maintain  
 > clear directions next to the object code saying where to find the

> Corresponding Source. Regardless of what server hosts the  
> Corresponding Source, you remain obligated to ensure that it is  
> available for as long as needed to satisfy these requirements.  
>  
> \*\*e)\*\* Convey the object code using peer-to-peer transmission, provided  
> you inform other peers where the object code and Corresponding  
> Source of the work are being offered to the general public at no  
> charge under subsection 6d.

A separable portion of the object code, whose source code is excluded from the Corresponding Source as a System Library, need not be included in conveying the object code work.

A “User Product” is either **(1)** a “consumer product”, which means any tangible personal property which is normally used for personal, family, or household purposes, or **(2)** anything designed or sold for incorporation into a dwelling. In determining whether a product is a consumer product, doubtful cases shall be resolved in favor of coverage. For a particular product received by a particular user, “normally used” refers to a typical or common use of that class of product, regardless of the status of the particular user or of the way in which the particular user actually uses, or expects or is expected to use, the product. A product is a consumer product regardless of whether the product has substantial commercial, industrial or non-consumer uses, unless such uses represent the only significant mode of use of the product.

“Installation Information” for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source. The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide support service, warranty, or updates for a work that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network may be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying.

#### Additional Terms

-----

“Additional permissions” are terms that supplement the terms of this

License by making exceptions from one or more of its conditions. Additional permissions that are applicable to the entire Program shall be treated as though they were included in this License, to the extent that they are valid under applicable law. If additional permissions apply only to part of the Program, that part may be used separately under those permissions, but the entire Program remains governed by this License without regard to the additional permissions.

When you convey a copy of a covered work, you may at your option remove any additional permissions from that copy, or from any part of it. (Additional permissions may be written to require their own removal in certain cases when you modify the work.) You may place additional permissions on material, added by you to a covered work, for which you have or can give appropriate copyright permission.

Notwithstanding any other provision of this License, for material you add to a covered work, you may (if authorized by the copyright holders of that material) supplement the terms of this License with terms:

```
> **a)** Disclaiming warranty or limiting liability differently from the
> terms of sections 15 and 16 of this License; or
>
> **b)** Requiring preservation of specified reasonable legal notices or
> author attributions in that material or in the Appropriate Legal
> Notices displayed by works containing it; or
>
> **c)** Prohibiting misrepresentation of the origin of that material, or
> requiring that modified versions of such material be marked in
> reasonable ways as different from the original version; or
>
> **d)** Limiting the use for publicity purposes of names of licensors or
> authors of the material; or
>
> **e)** Declining to grant rights under trademark law for use of some
> trade names, trademarks, or service marks; or
>
> **f)** Requiring indemnification of licensors and authors of that
> material by anyone who conveys the material (or modified versions of
> it) with contractual assumptions of liability to the recipient, for
> any liability that these contractual assumptions directly impose on
> those licensors and authors.
```

All other non-permissive additional terms are considered “further restrictions” within the meaning of section 1. If the Program as you received it, or any part of it, contains a notice stating that it is governed by this License along with a term that is a further restriction, you may remove that term. If a license document contains a further restriction but permits relicensing or conveying under this License, you may add to a covered work material governed by the terms of that license document, provided that the further restriction does not survive such relicensing or conveying.

If you add terms to a covered work in accord with this section, you must place, in the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms.

Additional terms, permissive or non-permissive, may be stated in the form of a separately written license, or stated as exceptions; the above requirements apply either way.

Termination

-----

You may not propagate or modify a covered work except **as** expressly provided under **this** License. Any attempt otherwise to propagate or modify it **is void**, and will automatically terminate your rights under **this** License (including any patent licenses granted under the third paragraph of section 11).

However, **if** you cease all violation of **this** License, then your license from a particular copyright holder **is** reinstated **“(a)”** provisionally, unless and until the copyright holder explicitly and finally terminates your license, and **“(b)”** permanently, **if** the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder **is** reinstated permanently **if** the copyright holder notifies you of the violation by some reasonable means, **this is** the first time you have received notice of violation of **this** License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under **this** section does not terminate the licenses of parties who have received copies or rights from you under **this** License. If your rights have been terminated and not permanently reinstated, you **do not** qualify to receive new licenses for the same material under section 10.

#### Acceptance Not Required for Having Copies

---

You are not required to accept **this** License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely **as** a consequence of using peer-to-peer transmission to receive a copy likewise does not **require** acceptance. However, nothing other than **this** License grants you permission to propagate or modify any covered work. These actions infringe copyright **if** you **do not** accept **this** License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of **this** License to **do** so.

#### Automatic Licensing of Downstream Recipients

---

Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject to **this** License. You are not responsible for enforcing compliance by third parties with **this** License.

An “entity transaction” **is** a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever licenses to the work the party's predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, **if** the predecessor has it or can **get** it with reasonable efforts.

You may not impose any further restrictions on the exercise of the rights granted or affirmed under **this** License. For example, you may not impose a license fee, royalty, or other charge for exercise of rights granted under **this** License, and you may not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that any patent claim **is** infringed by making, using, selling, offering for

sale, or importing the Program or any portion of it.

## Patents

A “contributor” is a copyright holder who authorizes use under this License of the Program or a work on which the Program is based. The work thus licensed is called the contributor's “contributor version”.

A contributor's “essential patent claims” are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a consequence of further modification of the contributor version. For purposes of this definition, “control” includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.

Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor's essential patent claims, to make, use, sell, offer for sale, import and otherwise run, modify and propagate the contents of its contributor version.

In the following three paragraphs, a “patent license” is any express agreement or commitment, however denominated, not to enforce a patent (such as an express permission to practice a patent or covenant not to sue for patent infringement). To “grant” such a patent license to a party means to make such an agreement or commitment not to enforce a patent against the party.

If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy, free of charge and under the terms of this License, through a publicly available network server or other readily accessible means, then you must either **(1)** cause the Corresponding Source to be so available, or **(2)** arrange to deprive yourself of the benefit of the patent license for this particular work, or **(3)** arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. “Knowingly relying” means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient's use of the covered work in a country, would infringe one or more identifiable patents in that country that you have reason to believe are valid.

If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of the covered work, then the patent license you grant is automatically extended to all recipients of the covered work and works based on it.

A patent license is “discriminatory” if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You may not convey a covered work if you are a party to an arrangement with a third party that is in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the parties who would receive the covered work from you, a discriminatory patent license **(a)** in connection with copies of the covered work

conveyed by you (or copies made from those copies), or **(b)** primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that may otherwise be available to you under applicable patent law.

#### No Surrender of Others' Freedom

---

If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not convey it at all. For example, if you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

#### Remote Network Interaction; Use with the GNU General Public License

---

Notwithstanding any other provision of this License, if you modify the Program, your modified version must prominently offer all users interacting with it remotely through a computer network (if your version supports such interaction) an opportunity to receive the Corresponding Source of your version by providing access to the Corresponding Source from a network server at no charge, through some standard or customary means of facilitating copying of software. This Corresponding Source shall include the Corresponding Source for any work covered by version 3 of the GNU General Public License that is incorporated pursuant to the following paragraph.

Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the work with which it is combined will remain governed by version 3 of the GNU General Public License.

#### Revised Versions of this License

---

The Free Software Foundation may publish revised and/or new versions of the GNU Affero General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU Affero General Public License “or any later version” applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the GNU Affero General Public License, you may choose any version ever published by the Free Software Foundation.

If the Program specifies that a proxy can decide which future versions of the GNU Affero General Public License can be used, that proxy's

public statement of acceptance of a version permanently authorizes you to choose that version for the Program.

Later license versions may give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version.

#### Disclaimer of Warranty

> THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY  
> APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT  
> HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY  
> OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO,  
> THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR  
> PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM  
> IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF  
> ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

#### Limitation of Liability

> IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING  
> WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS  
> THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY  
> GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE  
> USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF  
> DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD  
> PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS),  
> EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF  
> SUCH DAMAGES.

#### Interpretation of Sections \_Disclaimer of Warranty\_ and \_Limitation of Liability\_

If the disclaimer of warranty and limitation of liability provided above cannot be given local legal effect according to their terms, reviewing courts shall apply local law that most closely approximates an absolute waiver of all civil liability in connection with the Program, unless a warranty or assumption of liability accompanies a copy of the Program in return for a fee.

#### \_END OF TERMS AND CONDITIONS\_

#### How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively state the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

```
```plaintext
<one line to give the program's name and a brief idea of what it does.>
Copyright (C) <year> <name of author>
```

This program is free software: you can redistribute it and/or modify it under the terms of the GNU Affero General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Affero General Public License for more details.

You should have received a copy of the GNU Affero General Public License along with this program. If not, see <<http://www.gnu.org/licenses/>>.
```

Also add information on how to contact you by electronic and paper mail.

If your software can interact with users remotely through a computer network, you should also make sure that it provides a way for users to get its source. For example, if your program is a web application, its interface could display a “Source” link that leads users to an archive of the code. There are many ways you could offer source, and different solutions will be better for different programs; see section 13 for the specific requirements.

You should also get your employer (if you work as a programmer) or school, if any, to sign a “copyright disclaimer” for the program, if necessary. For more information on this, and how to apply and follow the GNU AGPL, see &lt;<http://www.gnu.org/licenses/>>&gt;.

```
// _____< END >- _____
// FILE NAME: : LICENSE.md
// TYPE : .MD
// PATH : LICENSE.md
// PROCESSED: 2026-09-06T21:55:35.363Z
// _____< END >- _____

// _____< START >- _____
// FILE NAME: : AGENTS.md
// TYPE : .MD
// PATH : AGENTS.md
// PROCESSED: 2026-09-06T21:55:35.363Z
// _____< START >- _____
AGENTS.md
```

Guidance for any coding agent (or human) working in this repo. Read this before making changes —it captures conventions, known gotchas, and the reasoning behind non-obvious structure so they don't get re-litigated or silently reverted.

## What this is

Pendex (`px`) is a CLI that compiles a project's source tree into banner-delimited `.txt` archives grouped by job category (source, web, style, terminal, configuration, documentation, testing, misc), and a `px split` command that reconstructs the original tree from those archives plus a manifest. Primary use case: packaging a project for LLM context windows. See `README.md` for the user-facing pitch and usage; this file is about \_working on\_ the codebase, not using it.

## Monorepo layout

Bun workspaces, strictly acyclic package graph:

```

```mermaid
@pendex/color →@pendex/theme →@pendex/core →@pendex/compile +
@pendex/split →root pendex CLI (src/)

@pendex/home (website, depends on nothing above)
```

- `@pendex/core` holds shared contracts: types, `ArchiveFormat`,
 `FileScanner`, `View`, `ConfigManager`, `bootstrap.resolveRunnerDeps()`.
- `@pendex/compile` and `@pendex/split` each have standalone entry
 points (`Compile.ts` / `Split.ts`) that resolve their own deps via
 `@pendex/core`'s `bootstrap.ts` —they do not depend on the root
 shell package. This lets them run outside the interactive menu.
- `@pendex/home` is the whole website (marketing home page and the
 user guide + generated API reference, merged into one package —see
 "The website" below). It has no relationship to the CLI packages
 other than living in the same workspace.
- Cross-package round-trip tests live at the repo root
 (`tests/integration.test.ts`).
- Root `src/` is the CLI shell: `App.ts` (menu orchestrator), `commands/`
 (thin containers), `components/`, `utils/`.

Never introduce a cycle. If you find yourself importing from a
package "above" the current one in that chain, the code belongs
somewhere else —usually `@pendex/core`.

Architecture pattern: core = hooks, views = components, commands = containers

```

Borrow the React mental model even though **this is** a CLI:

```

- **Core / Service** layer = pure logic (hooks). No `@clack/prompts`
 calls, no console output. Testable without mocking a terminal.
- **View** layer = the thing that owns a full `@clack/prompts` session,
 intro through outro. Every exit path (success, error, early
 return) must close the session it opened. A view that opens `intro()`
 but leaves `outro()` to its caller is a bug waiting to happen —see
 "Bugs found and fixed" below for what that looked like in practice.
- **Command** layer = thin wiring between a menu entry and a view/service.
 If a command is doing real logic, that logic belongs one layer down.

```

Don't split into command + service + view **for** something trivial. `Exit`
**is** deliberately a single file —one `outro()` and a `process.exit()`
don't earn three files.

## The archive format

`ArchiveFormat.ts`'s `parseArchive()` **is** a path-matched frame state
machine. Content **is** preserved verbatim between banner frames —
byte-perfect round trips including trailing newlines and empty files.
Inner **frames** (an archive nested inside another archive's content, e.g.
when self-hosting) carry inner paths and cannot falsely terminate outer
entries. If you touch **this** file, the round-trip and self-hosting tests
**in** `packages/core/tests/archiveformat.test.ts` and
`tests/integration.test.ts` are the ones that matter most —verbatim
byte-fidelity **is** the entire point **of** the format.

## Themes

```

- `packages/theme/themes/pendex.toml` is the **canonical** theme. Every
 other theme file (`dracula.toml`, `tokyonight.toml`, `onedark.toml`,
 `monokaipro.toml`) must mirror its exact section layout —same tables,
 same keys, different values.
- A theme name is the file stem under `packages/theme/themes/` (e.g.

```

- `"pendex"`, `"dracula"`) —not a dark/light binary. `ThemeManager`` degrades to the built-in brand palette for unknown/missing/malformed themes rather than crashing; the theme is cosmetic and must never take the app down.
- **\*\***The shipped default config's `theme`` value must name a real file in `packages/theme/themes/``. **\*\*** It was `"default"` for a long time with no `default.toml`` on disk, which silently degraded every fresh install to the fallback palette instead of loading `pendex.toml`` — caught only because a test asserted on it (`packages/core/tests/bootstrap.test.ts``). Don't reintroduce a theme name that doesn't correspond to a file.
- `[brand]`` table's Category-named keys (`source``, `web``, `style``, `terminal``, `configuration``, `documentation``, `testing``, `misc``) drive per-job-category color-coding via `categoryStyle()`` on the `View`` base class. `bootstrap.ts``'s `extractCategoryColors()`` only keeps keys that are literally one of the eight `Category`` values —abbreviating a key (e.g. `config`` instead of `configuration``) silently drops it, it doesn't error.
- All palette colors are WCAG-verified against `#1A1A1A`` and `#262626`` backgrounds. Keep new colors verified the same way.

## The website (`@pendex/home``)

One Vite package, one build, two pages, sharing one `pendex`` daisyUI theme (`src/index.css``):

- **\*\***/`**` —marketing home page. `index.html`` → `src/main.tsx`` → `src/App.tsx``.
- **\*\***/`docs/``**\*\*** —user guide + generated API reference. `docs/index.html`` → `docs/main.tsx`` → `src/docs/DocsApp.tsx``. The API reference itself (`/docs/api/``) is TypeDoc output written to `public/docs/api/``. It's committed to the repo (not gitignored, not touched by `bun run clean``) so the site has a working `/docs/api`` even without a fresh TypeDoc run; it's copied into `dist/docs/api/`` automatically because it lives under `publicDir``. **\*\***You don't need to regenerate and commit this by hand**\*\*** —`.github/workflows/deploy.yml``'s "Sync generated API docs" step does it on every push to `main``: it rebuilds the site (which regenerates `public/docs/api/`` as a side effect), and if that output actually differs from what's committed, commits and pushes it back with `[skip ci]`` (so it doesn't re-trigger itself). TypeDoc's output is byte-for-byte deterministic given identical source, so this commit only ever happens when a library package's real JSDoc content changed —never as noise. If you want a fresh copy locally before committing something else, `bun run --cwd packages/home docs:api``.

This used to be two separate workspace packages (`@pendex/home`` and `@pendex/api-docs``) with two Vite configs, deployed together via a "combine builds" step in CI. They were merged into one package with one `vite.config.ts`` (living in `packages/home/``, not the repo root) and a two-entry multi-page build. If you're tempted to split the docs back out into their own package, don't —the whole point of the merge was one theme, one dependency set, one build pipeline, no drift between two copies of the same CSS.

**\*\***`typescript`` is pinned to `5.9.3`` inside `packages/home/package.json``, overriding the workspace's `7.0.2``, `_only_` because TypeDoc has no TS 7 support yet (its own tracking issue says the TS7 API is a ground-up rewrite with no timeline). Bun nests this version locally rather than hoisting it, so it doesn't affect any other package. Remove the pin once TypeDoc supports TS 7 —check `typedoc``'s peer dependency range before bumping it back.

**\*\***Declaration output (`tsc -b``) is redirected to `node_modules/.tmp/types/`` via `declarationDir`` in `tsconfig.app.json`` / `tsconfig.node.json``.

Without it, every `.tsx/.ts` file gets a same-named `.d.ts` sibling dumped straight into `src/` on every build —annoying, and it used to happen silently until lint/format started flagging the generated files as unformatted source. If you see stray `.d.ts` files next to source after a build, something regressed [this](#) setting.

## ## Tooling

- **Package manager** / runtime: Bun only. **Built-in** Bun APIs in `src/` (`Bun.file`, `Bun.write`, `Bun.$`, `Bun.TOML.parse`), no Node-only APIs. `devEngines` in the root `package.json` enforces [this](#).
- **TypeScript strict mode**, no `any` —enforced everywhere except two explicitly out-of-scope standalone [utilities](#) (`src/utils/HeaderComments.ts`, `src/utils/update-config.ts`, see their `coveragePathIgnorePatterns` entry in `bunfig.toml`).
- **Lint/format**: Oxlint + Oxfmt, not ESLint/Prettier. **Configs** are `.oxlintrc.json` and `.oxfmt.json` at the repo root. `bun run lint` runs `oxlint --fix` then `oxfmt`; `bun run lint:check` / `bun run format:check` run non-mutating versions for CI. ESLint was removed because `typescript-eslint` doesn't support TypeScript 7 yet (same upstream-lag problem as TypeDoc above) —Oxlint has no such gap and is also just faster. Don't reintroduce ESLint/Prettier config files; if a new rule is needed, add it to `.oxlintrc.json`.
- **Docs**: TypeDoc, driven by `typedoc.json` + `tsconfig.typedoc.json` at the repo root, entry points across all five library [packages](#) (`color`, `theme`, `core`, `compile`, `split` —not `home`, which isn't a library). Output goes to `packages/home/public/docs/api`.

## ## Commands that matter

```
```sh
bun install          # from repo root
bun run typecheck    # tsc --noEmit at root + every package's
own typecheck
bun run test         # bun test at root (CLI shell) + every
package's own tests
bun run lint         # oxlint --fix . ; oxfmt .
bun run build        # builds the website (packages/home) —
home page + docs + API reference
bun run build:pendex # typecheck, then compile the CLI to ./dist/px
bun run dev          # website dev server (both / and /docs)
bun run clean        # removes build output (dist, coverage,
generated API docs, etc.)
bun run start        # run the interactive CLI shell directly
```
```

**When you touch `bun --filter` in any script**: never combine it with `run`. `bun --filter '<pattern>' run <script>` silently matches zero packages in this Bun version ("No packages matched the filter") and exits non-zero, while `bun --filter '<pattern>' <script>` (no `run`) works correctly. This bit us for a long time: the root `test` script's second half (meant to run every package's own tests) was silently failing before it ever reached the packages, which hid two real test failures in `@pendex/core` (the `theme-default` bug above) for who knows how long. If a `bun --filter` invocation you add starts reporting zero matched packages, [this is](#) almost certainly why —drop the `run`.

## ## Testing conventions

- `tests/preload.ts` (root) mocks `@clack/prompts` globally so `intro()`/`outro()`/`select()`/etc. never hang waiting for real terminal input in tests; `tasks()` still runs each task function for real so service-layer work under test actually executes.

- Each `package` scopes its own ``bun test`` discovery via ``root = "."`` in its own ``bunfig.toml`` (``root = "tests"`` at the repo root). Without this, ``bun run --filter '*' test`` re-runs the `_entire_` workspace suite from inside every `package` and collides on shared sandbox directories —this happened before all six ``bunfig.toml`` files had ``root`` scoping (``@pendex/theme`` and ``@pendex/color`` were missing it). If you add a new `package`, give it a ``bunfig.toml`` with ``root = "."``.
- ``Constants.BASE_DIR`` / ``GITIGNORE_PATH`` / ``RUNTIME_CONFIG_PATH`` are `**live` getters over ``process.cwd()``, not fields frozen at module-first-import. This is what makes ``process.chdir()`` in a test's ``beforeEach`` actually work for sandboxing —don't turn them back into plain ``readonly`` fields computed once.
- Cross-package round-trip fidelity (compile →split byte-perfect, including empty directories) is covered in ``tests/integration.test.ts`` at the repo root, not inside either `package` —it's testing the contract `_between_` ``@pendex/compile`` and ``@pendex/split``, not either one in isolation.

## ## Dependency policy

- Runtime deps that matter and should stay pinned to roughly these major/minor lines unless there's a specific reason to move: ``typescript@^7.0.2``, ``@clack/prompts@^1.7.0``, ``tailwindcss@^4.3.3``, ``daisyui@^5.7.9``, ``react@^19.2.8``, ``react-dom@^19.2.8``.
- Dev tooling: ``@tailwindcss/vite``, ``@types/bun``, ``@types/react``, ``@types/react-dom``, ``@vitejs/plugin-react``, ``vite``, ``oxlint``, ``oxfmt``.
- PDF export and syntax highlighting are `**planned` but not wired up yet. Their dependencies (``@pdf-lib/fontkit``, ``pdf-lib``, ``prismjs``, ``@types/prismjs``) live in root ``optionalDependencies`` specifically so ``bun install`` doesn't fail if one of them can't build in some environment, and specifically so nothing in ``src/`/`packages*/src`` imports them yet. Move them to ``dependencies`` when that feature actually starts, not before.
- Before adding a new tool (linter, doc generator, formatter, etc.), check its TypeScript peer-dependency range against ``^7.0.2`` first. This repo has hit the "tool doesn't support TS7 yet" wall twice already (ESLint's ``typescript-eslint``, TypeDoc) —it's a live upstream gap across the ecosystem right now, not a one-off.
- Don't add a dependency "just in case." ``trash`` and ``concurrently`` were both in ``package.json`` for a long time with zero references anywhere in the codebase before being removed —if you add something, use it in the same change.

## ## Known-fixed bugs worth knowing about (don't reintroduce)

- ``theme.bold(theme.warning(x))`` chaining a not-yet-coerced themed value into another chain call throws ``Symbol.toPrimitive returned an object``. Force an intermediate template literal (```${...}```) before passing a themed value into another theme call.
- ``Bun.Glob('**/')`` doesn't surface directories whose entire subtree has zero files. ``findEmptyDirectories`` in ``FileScanner.ts`` uses ``**/*`` with ``onlyFiles: false`` / ``onlyFiles: true`` and a `set` difference instead —don't swap it back to a trailing-slash glob.
- ``(this.state.exit() ?? process.exit(0))`` calls ``exit()`` immediately (with no args) and, being `void`-returning, `_also_` always fires the real ``process.exit(0)`` via ``??`` —even with a test mock injected. ``??`` needs to pick a `function` `**reference**`, not a call result: ``(this.state.exit ?? process.exit)(0)``.
- ``(value as any).__clackResult = x`` where ``value`` is a string primitive is a silent no-op —primitive property writes don't error, they just do nothing. Don't mutate primitives and expect it to stick.
- ``parsePendexTheme``'s table extraction must reject arrays explicitly —``typeof [] === 'object`` in JS, so an unguarded check turns a TOML

array under a table key into a bogus `{0: ...}` object instead of  
`undefined`.

- `Exit.ts` used to hand-draw its goodbye message with raw  
`console.log('|')` / `console.log('L ' + msg)` instead of using  
`@clack/prompts`' `outro()` —reinventing clack's own box-drawing  
and leaving the shell's clack session technically still open. It now  
calls `outro()` like every other view.

## Before you commit

1. `bun run typecheck` —zero diagnostics, every package.
2. `bun run test` —every package's tests pass (not just the root half;  
see the `--filter`/`run` warning above if the workspace half seems  
to skip silently).
3. `bun run lint` (or `lint:check` + `format:check` if you don't want  
auto-fixes applied).
4. If you touched `packages/home`: `bun run build` and spot-check  
`packages/home/dist/` has both `index.html` and `docs/index.html`,  
and that `docs/api/` has content.

```
// _____< END >- _____
// FILE NAME: : AGENTS.md
// TYPE : .MD
// PATH : AGENTS.md
// PROCESSED: 2026-09-06T21:55:35.363Z
// _____< END >- _____
```